

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

**«Санкт-Петербургский политехнический университет Петра
Великого»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

02.03.01 Математика и компьютерные науки

Отчет
Структуры данных
Летняя научно-исследовательская работа

Выполнил:

студент группы 5130201/50301 _____ Ташлыкова Е.В.

Принял:

канд. техн. наук., доцент ВШТИИ _____ Глазунов В.В.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	5
2 Особенности реализации	6
2.1 Установка и настройка среды виртуализации	6
2.2 Работа в командном интерпретаторе	8
2.3 Разработка статических и динамических библиотек	11
2.4 Исследование разделяемых библиотек	14
2.5 Двоичные файлы и визуализация в псевдографике	16
3 Результаты работы и эксперименты	27
Заключение	34
Список используемых источников	37
А. Исходный код файлов задания №1	38
В. Исходный код файлов задания №2	54

Введение

В рамках летней научно-исследовательской практики была выполнена работа по освоению операционной системы GNU/Linux Debian 12, инструментов разработки и систем программирования на языке Си. Основное внимание уделялось работе в среде виртуализации, изучению командного интерпретатора, а также разработке программного обеспечения с использованием статических и динамических библиотек, работе с двоичными файлами и псевдографической библиотекой ncurses.

В работе использовались следующие технологии и инструменты:

- VirtualBox – среда виртуализации для запуска гостевой операционной системы;
- GNU/Linux Debian 12 – операционная система, установленная на виртуальную машину;
- SSH – протокол для удаленного подключения к терминалу;
- CMake – мета-система сборки для управления процессом компиляции;
- ncurses – библиотека для создания псевдографических интерфейсов;
- zlib – библиотека для вычисления контрольной суммы CRC-32;
- GCC – компилятор языка Си;
- Make – система сборки;
- Nano и Visual Studio Code – редакторы кода.

Практика направлена на закрепление навыков работы в Linux-среде, разработки на языке Си, управления зависимостями и сборкой проектов, а также

на исследование производительности программ при различных подходах к линковке.

1 Постановка задачи

Для того, чтобы считать работу выполненной необходимо закрыть следующие задачи:

1. Установить и настроить виртуальную среду Debian 12.14;
2. Обучиться основам работы в командном интерпретаторе;
3. Разработать функцию проверки слова на палиндром в итеративном и рекурсивном вариантах;
4. Сборка статической и динамической библиотек, реализация функции через `dlopen`;
5. Провести тестирование скорости работы программы при различных флагах оптимизации (`O0`, `O1`, `O2`, `Os`), сравнить размер результирующих файлов;
6. Провести исследование зависимостей с `ldd`;
7. Создать структуры `PRICE` и заголовка базы данных;
8. Создать функции поиска средней цены товара с максимальным количеством в заданной категории;
9. Визуализировать структуры в псевдографике `ncurses`, провести тестирование работоспособности;
10. Описать проделанную работу в итоговом отчете.

2 Особенности реализации

2.1 Установка и настройка среды виртуализации

VirtualBox – это кроссплатформенное программное обеспечение для виртуализации, которое позволяет запускать несколько операционных систем (ОС) одновременно на одном физическом компьютере. В качестве ISO-образа использовался *debian – 12.14.0 – arm64 – netinst*.

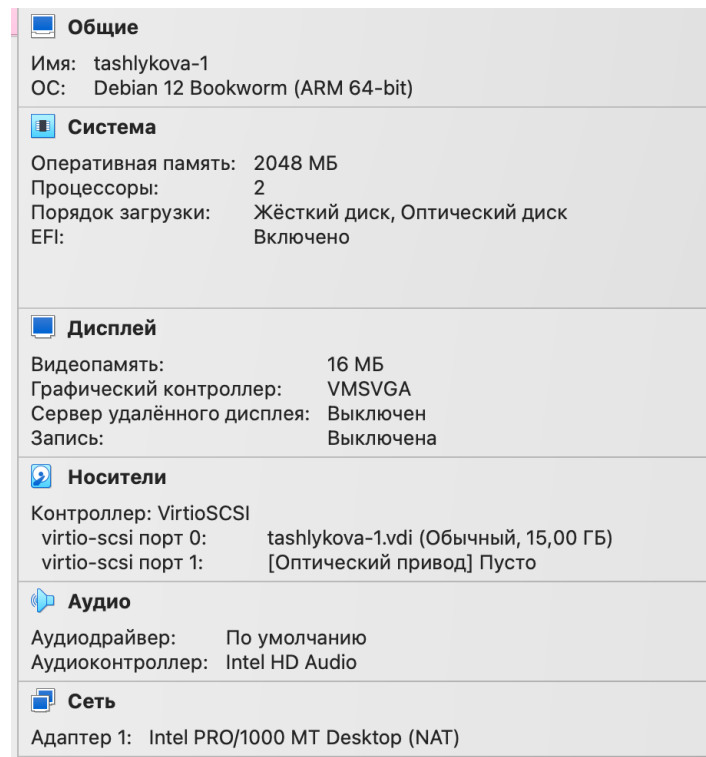


Рис. 1: Детали виртуальной машины

После загрузки системы на терминале `tty1` отображается приглашение к вводу имени пользователя и пароля.

```
Debian GNU/Linux 12 tashlykova-1 tty1
tashlykova-1 login: tashlykova
Password:
Linux tashlykova-1 6.1.0-49-arm64 #1 SMP Debian 6.1.174-1 (2026-05-26) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Jun 10 17:07:40 MSK 2026 on tty1
tashlykova@tashlykova-1:~$ _
```

Рис. 2: Приглашение tty1

Также в виртуальной машине была выполнена настройка проброса портов (Port Forwarding):

- Тип подключения: NAT
- Протокол: TCP;
- Адрес хоста: 127.0.0.1;
- Порт хоста: 2222;
- Порт гостя: 22.

Это позволило подключаться к гостевой ОС через SSH.

Имя	Протокол	Адрес хоста	Порт хоста	Адрес гостя	Порт гостя
tcp_2222_22	TCP	127.0.0.1	2222		22

Рис. 3: Проброс портов

Подключение к виртуальной машине выполнено с помощью встроенного SSH-клиента. Для беспарольной SSH-аутентификации необходимо сгенерировать пару ключей и публичный ключ скопировать на виртуальную машину.

```
ssh-keygen -t rsa -b 4096
ssh-copy-id -p 2222 tashlykova@127.0.0.1
ssh -p 2222 tashlykova@localhost
```

Благодаря этим действиям подключение к ВМ осуществляется без ввода пароля.

2.2 Работа в командном интерпретаторе

Интерпретатор предоставляет большое количество команд для работы. С помощью *man* я ознакомилась со справкой команд из списка задания, они приведены в таблице [1](#).

Таблица. 1: Основное назначение команд

Команда	Пояснение
whoami	Вывод имени текущего пользователя
date	Вывод текущей даты и времени
uptime	Время работы системы и нагрузка
df	Информация о свободном месте на дисках
cp	Копирование файлов
mv	Перемещение/переименование файлов
touch	Создание пустого файла
rm	Удаление файлов
stat	Информация о файле (размер, права, время)
file	Определение типа файла
find	Поиск файлов по критериям
cat	Вывод содержимого файла
hexdump	Просмотр файла в шестнадцатеричном виде
less	Постраничный просмотр файла
time	Измерение времени выполнения команды


```

[tashlykova@tashlykova-1:~/task1/build$ whoami
tashlykova
[tashlykova@tashlykova-1:~/task1/build$ cd ..
[tashlykova@tashlykova-1:~/task1$ cd ..
[tashlykova@tashlykova-1:~$ whoami
tashlykova
[tashlykova@tashlykova-1:~$ date
Sat Jun 20 03:41:15 PM MSK 2026
[tashlykova@tashlykova-1:~$ uptime
 15:41:34 up 19:16,  2 users,  load average: 0.00, 0.00, 0.00
[tashlykova@tashlykova-1:~$ df
Filesystem      1K-blocks      Used Available Use% Mounted on
udev              960388          0    960388   0% /dev
tmpfs             202300          504    201796   1% /run
/dev/sda2        13873468 2187960  10958980  17% /
tmpfs            1011488          0    1011488   0% /dev/shm
tmpfs              5120          0         5120   0% /run/lock
/dev/sda1         524008         6208    517800   2% /boot/efi
tmpfs            202296          0    202296   0% /run/user/1000
[tashlykova@tashlykova-1:~$ touch test.c
[tashlykova@tashlykova-1:~$ ls
task1 task2 test.c tree.sh
[tashlykova@tashlykova-1:~$ rm test.c
[tashlykova@tashlykova-1:~$ ls
task1 task2 tree.sh
[tashlykova@tashlykova-1:~$ whereis main.c
[tashlykova@tashlykova-1:~$ cat task1/src/palindrome.h
// palindrome.h
#ifndef PALINDROME_H
#define PALINDROME_H

#include <stdbool.h>

bool palindrome(char *string, int n);
bool palindrome_recursive(char *string, int n);
int string_len(char *string);

```

Рис. 4: Работа в командном интерпретаторе

С помощью *history* можно просмотреть всю историю команд. Зафиксируем три последние:

```

chmod +x scripts/*.sh
./scripts/analyze_libs.sh
./scripts/test_optimization.sh

```

Далее было необходимо исследовать зависимости между программой командного интерпретатора */bin/bash* и динамическими библиотеками операцион-

ной системы при помощи команды `ldd`. Для этого построим рекурсивное дерево зависимостей. Для наглядности был разработан скрипт *tree.sh*.

```
# tree.sh
function print_deps() {
    local file=$1
    local indent=$2
    local first=$3
    if [ ! -f "$file" ]; then
        echo "${indent}[НЕ НАЙДЕНО: $file]"
        return
    fi
    ldd "$file" 2>/dev/null | while read line; do
        if echo "$line" | grep -q "linux-vdso"; then
            continue
        fi
        if echo "$line" | grep -q "=> /"; then
            lib=$(echo "$line" | awk '{print $3}')
            name=$(basename "$lib")
            echo "${indent}|-- $name"
            print_deps "$lib" "${indent}|" "no"
        elif echo "$line" | grep -q "^/lib/ld-linux"; then
            # Загрузчик (ld-linux)
            lib=$(echo "$line" | awk '{print $1}')
            name=$(basename "$lib")
            echo "${indent}|__ $name"
            # У загрузчика нет зависимостей, но можем проверить
            print_deps "$lib" "${indent}" "no"
        fi
    done
}
print_deps "$1" "" "yes"
```

Все программы зависят от `libc.so.6` – стандартной библиотеки языка Си. Больше всего зависимостей у *bin/ls*.

```

[tashlykova@tashlykova-1:~$ ./tree.sh /bin/bash
└─ libtinfo.so.6
   └─ libc.so.6
└─ libc.so.6
[tashlykova@tashlykova-1:~$ ./tree.sh /bin/ls
└─ libselinux.so.1
   └─ libpcre2-8.so.0
      └─ libc.so.6
└─ libc.so.6
└─ libc.so.6
└─ libpcre2-8.so.0
   └─ libc.so.6
[tashlykova@tashlykova-1:~$ ./tree.sh /usr/bin/grep
└─ libpcre2-8.so.0
   └─ libc.so.6
└─ libc.so.6
tashlykova@tashlykova-1:~$ █

```

Рис. 5: Деревья зависимостей

2.3 Разработка статических и динамических библиотек

В индивидуальном задании №1 было необходимо создать библиотеку с реализацией функции, которая проверяет является ли указанное слово палиндромом, при этом она должня быть реализована различными способами: с рекурсией и без рекурсии, также в реализации запрещалось использовать библиотечные функции. Для этого я написала две основные функции *palindrome*, *palindrome_recursive* – итеративная и рекурсивная реализация, а так же одну вспомогательную функцию *string_len*, для вычисления длины строки.

```

#ifdef PALINDROME_H
#define PALINDROME_H

#include <stdbool.h>

bool palindrome(char *string, int n);
bool palindrome_recursive(char *string, int n);
int string_len(char *string);

```

#endif

Алгоритм *palindrome*: сравниваются символы с начала и конца строки, постепенно сдвигаясь к центру. При обнаружении несовпадающей пары возвращается false. Время работы $O(n/2)$, память $O(1)$.

Алгоритм *palindrome_recursive*: базовый случай – пустая строка или строка из одного символа являются палиндромами. Иначе сравниваются первый и последний символы строки; при совпадении выполняется рекурсивный вызов для подстроки без крайних символов. Время работы $O(n)$, память $O(n)$.

Исходный код приведен в Приложении А.

Для управления сборкой библиотек и тестовых программ использовалась мета-система сборки CMake. Структура проекта выглядит следующим образом:

```
task1
----CMakeLists.txt
----src/
    |---CMakeLists.txt
    |---palindrome.c
    |---palindrome.h
----tests/
    |---CMakeLists.txt
    |---test_palindrome.c
    |---test_dlopen.c
----scripts/
    |---analyze_libs.sh
    |----test_optimization.sh
```

Статическая библиотека – это коллекция объектных файлов (.o), объеди-

ненных в один архивный файл. Во время компиляции программы, использующей статическую библиотеку, компоновщик (linker) копирует весь необходимый код из библиотеки в исполняемый файл. Таким образом, исполняемый файл становится самодостаточным и не зависит от наличия библиотеки в системе во время выполнения.

Динамическая (разделяемая) библиотека – это коллекция скомпилированного кода, который загружается в память программы во время запуска. Исполняемый файл содержит только ссылки на функции и данные этой библиотеки, а не их копии. Это позволяет нескольким программам использовать одну и ту же копию библиотеки в памяти.

Настройка сборки библиотек в *src/CMakeLists.txt*:

```
# Статическая библиотека
add_library(palindrome_static STATIC ${LIB_SOURCES})
set_target_properties(palindrome_static PROPERTIES
    OUTPUT_NAME palindrome
)

# Динамическая библиотека
add_library(palindrome_shared SHARED ${LIB_SOURCES})
set_target_properties(palindrome_shared PROPERTIES
    OUTPUT_NAME palindrome
    VERSION ${PROJECT_VERSION}
    SOVERSION 1
)
```

При обычной компоновке с динамической библиотекой, ОС автоматически загружает все необходимые библиотеки при запуске программы. Если какая-либо из них отсутствует, программа не запустится.

Однако существует еще более гибкий механизм работы с динамическими библиотеками – их динамическая загрузка во время выполнения (runtime

loading). Runtime loading предоставляет программисту полный контроль над процессом, это означает, что программа может решить, какую библиотеку загрузить и какие функции из нее использовать, не на этапе компиляции, а уже в процессе своей работы. Для реализации этого механизма в Unix/Linux используется набор функций, предоставляемых библиотекой dl (dynamic linking library)

Для загрузки библиотеки во время выполнения использованы функции `dlopen()` и `dlsym()`:

```
void *handle = dlopen("libpalindrome.so", RTLD_LAZY);
palindrome_func_t func = (palindrome_func_t)dlsym(handle, "palindrome");
```

2.4 Исследование разделяемых библиотек

Как происходит тестирование функций *palindrome* и *palindrome_recursive*:

1. Генерируются случайные строки заданной длины;
2. Для каждой строки вычисляется длина с помощью *string_len*;
3. Выполняется проверка на палиндром (итеративная или рекурсивная);
4. Измеряется процессорное время с помощью *clock()*;
5. Результат усредняется по количеству итераций.

Тестирование проходит для каждого типа линковки библиотеки. Также дополнительно разработан скрипт для автоматического тестирования всех флагов оптимизации:

- -O0 – без оптимизации (режим отладки)
- -O1 – базовые оптимизации
- -O2 – полная оптимизация (рекомендуемый уровень)

- -Os – оптимизация по размеру кода

Исследуем разделяемые библиотеки в полученной программе, используя команду `ldd` и построим соответствующие рекурсивные деревья зависимостей.

```
[tashlykova@tashlykova-1:~$ ./tree.sh task1/build_analysis/bin/test_palindrome
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1

[tashlykova@tashlykova-1:~$ ./tree.sh task1/build_analysis/bin/test_palindrome_dynamic
├── libpalindrome.so.1
│   ├── libc.so.6
│   │   └── ld-linux-aarch64.so.1
│   └── ld-linux-aarch64.so.1
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1

[tashlykova@tashlykova-1:~$ ./tree.sh task1/build_analysis/bin/test_dlopen
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1

[tashlykova@tashlykova-1:~$ ./tree.sh task1/build_analysis/lib/libpalindrome.so
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1
```

Рис. 6: Дерево зависимостей

Статическая линковка (*test_palindrome*)

Программа содержит только 2 зависимости: `libc.so.6` (стандартная библиотека Си) и `ld-linux-aarch64.so.1` (динамический загрузчик). Библиотека `libpalindrome` не отображается в зависимостях, так как ее код был скопирован в исполняемый файл на этапе компиляции.

Динамическая линковка (*test_palindrome_dynamic*)

Программа содержит 3 зависимости: `libc.so.6`, `ld-linux-aarch64.so.1`, `libpalindrome.so.1` (динамическая библиотека с функциями палиндрома). Библиотека `libpalindrome` загружается при запуске программы. Если библиотека не будет найдена в системе, программа не запустится.

Динамическая загрузка через `dlopen` (*test_dlopen*)

Программа содержит только 2 зависимости: `libc.so.6` и `ld-linux-aarch64.so.1`. Отметим, что `libpalindrome.so` не отображается в выводе `ldd`, так как загружается динамически во время выполнения через функции `dlopen()` и `dlsym()`. Это позволяет программе загружать библиотеку по требованию и обрабатывать ситуацию ее отсутствия.

2.5 Двоичные файлы и визуализация в псевдографике

Следующий этап научно-исследовательской работы – разработка программы с использованием псевдографической библиотеки `ncurses`.

`Ncurses` («new curses») – это библиотека предназначенная для управления вводом-выводом на терминал. Она позволяет работать с виртуальным экраном, создавать и управлять несколькими окнами на одном терминале, каждое со своим содержимым и атрибутами.

В рамках задания было необходимо реализовать структуру с именем `PRICE`, содержащую следующие поля:

- Название категории (строка `Си`).
- Название товара (строка `Си`).
- Стоимость товара в рублях (вещественный тип).
- Количество товаров в наличии (целочисленный тип).

А так же создать структуру описывающую заголовок базы данных, содержащую:

- Сигнатура формата файла (4 байта) – содержит первые 4 буквы фамилии

латиницей.

- Номер транзакции (4 байта) – целое число, которое должно увеличиваться при каждом обращении к файлу, как при чтении, так и при записи.
- Количество структур в файле (4 байта) – количество структур данных из п.1 следующих за заголовком.
- Контрольная сумма (4 байта) – контрольная сумма всех данных следующих за заголовком по алгоритму CRC-32 вызываемая из библиотеки zlib.h

Далее представлена структура *PRICE*, а так же структура для заголовка базы данных *DB_HEADER*.

```
typedef struct
{
    char *category;
    char *good;
    double price;
    unsigned int num;
} PRICE;

typedef struct
{
    char signature[4];
    unsigned int tr_num;
    unsigned int st_count;
    unsigned int crc32;
} DB_HEADER;
```

Программа на ncurses

В начале программы нас приветствует меню «product database management» состоящее из 6 полей, при вводе числа от 1 до 6 можно выполнить одну из команд.

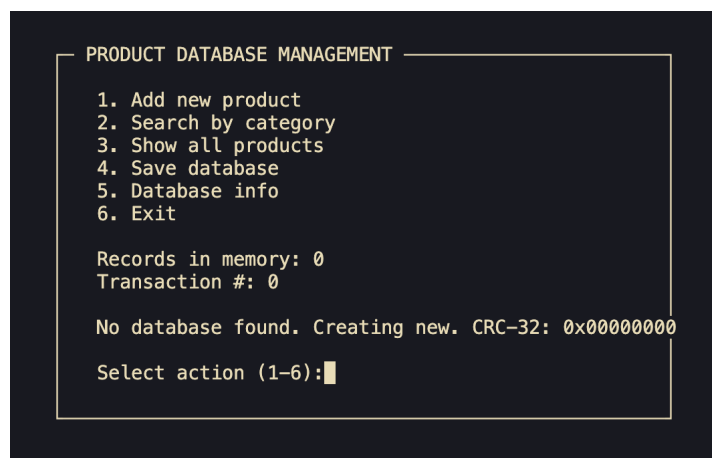


Рис. 7: Стартовое меню

Чтобы добавить новый продукт необходимо ввести цифру **1**. Поля появляются в окне по мере заполнения данных пользователем. При этом, в любой момент можно нажать кнопку ESC на клавиатуре и запись данных прекратится.

```
ENTER PRODUCT DATA
Category:  milk
Product name: 
Press ESC to cancel

ENTER PRODUCT DATA
Category:  milk
Product name: cheese
Cost (rub): 12.56
Amount: 10
Press ESC to cancel
```

Рис. 8: Добавление нового продукта

Если все данные введены корректно, то пользователь получает сообщение, что новый продукт успешно записан в базу.

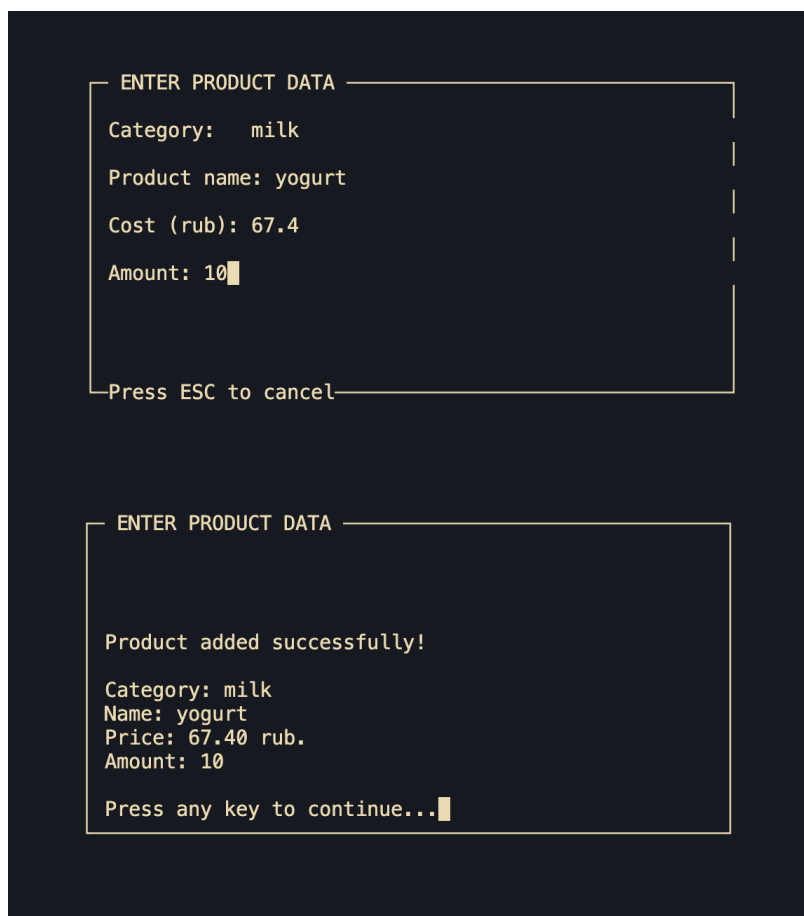


Рис. 9: Добавление нового продукта

После записи пользователь возвращается в исходное меню, теперь введем цифру **3**, чтобы убедиться, что все продукты были добавлены.

```
ALL PRODUCTS
[1] Category: milk
    Name: cheese
    Price: 12.56 rub.
    Amount: 10
[2] Category: milk
    Name: yogurt
    Price: 67.40 rub.
    Amount: 10

Total: 2 products. Press any key...
```

Рис. 10: Просмотр всех продуктов в базе

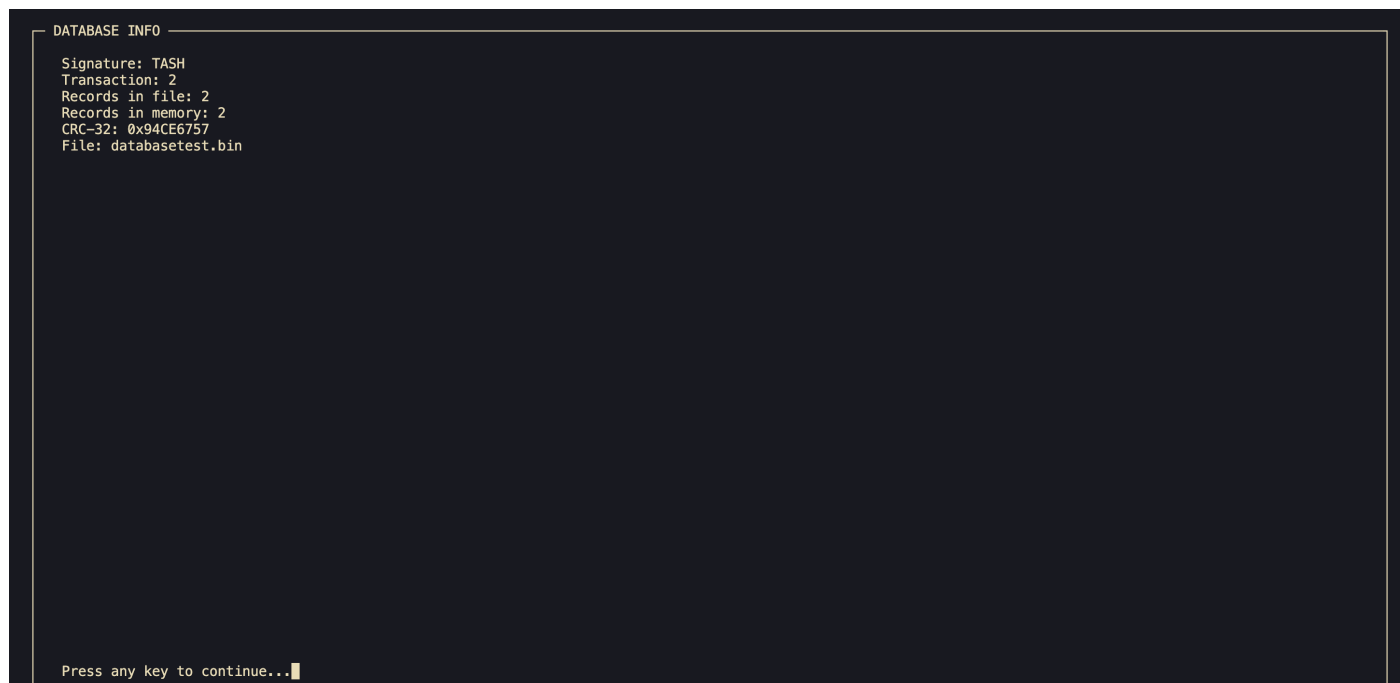
При вводе цифры 4 можно сохранить базу данных, после этого значение контрольной суммы CRC-32 изменится.

```
Database saved successfully!
Records: 2
CRC-32: 0x94CE6757

Press any key...
```

Рис. 11: Сохранение базы данных

Чтобы просмотреть больше информации о базе данных, необходимо ввести цифру **5**. Это позволит узнать сигнатуру формата файла, номер транзакции, количество структур в файле, количество записей, которые загружены в оперативную память, контрольную сумму, а также файл, в котором хранятся данные.

A screenshot of a terminal window with a dark background. The text is white and shows database information. At the top, it says 'DATABASE INFO' followed by a horizontal line. Below the line, the following information is displayed: 'Signature: TASH', 'Transaction: 2', 'Records in file: 2', 'Records in memory: 2', 'CRC-32: 0x94CE6757', and 'File: databasetest.bin'. At the bottom of the terminal, it says 'Press any key to continue...' followed by a cursor icon.

```
DATABASE INFO
-----
Signature: TASH
Transaction: 2
Records in file: 2
Records in memory: 2
CRC-32: 0x94CE6757
File: databasetest.bin

Press any key to continue...
```

Рис. 12: Просмотр дополнительной информации о базе данных

Перейдем к следующему индивидуальному заданию, в котором необходимо найти и вывести информацию о средней цене товаров с максимальным количеством, в категории, значение которой введено с клавиатуры. Для этого введем **2** в основном меню программы. Далее появится окно для ввода категории и отдельный дисплей для вывода результата поиска.

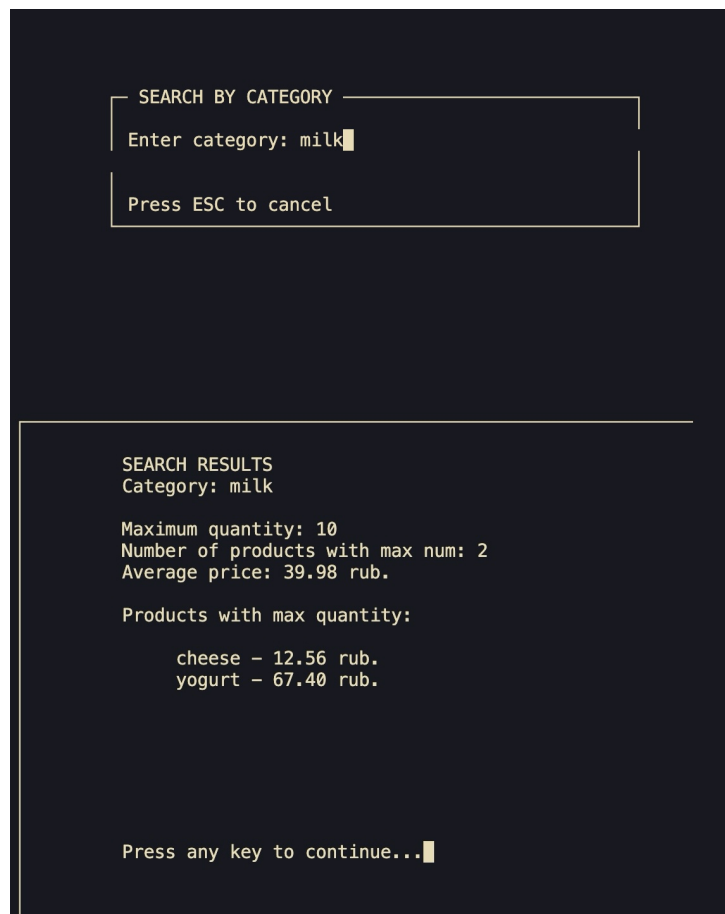


Рис. 13: Поиск по существующей категории

Если такого товара нет, на дисплее выдается соответствующее сообщение. Так же, как и во время добавления новой структуры, во время поиска можно нажать кнопку ESC, чтобы в любой момент выйти в основное меню.

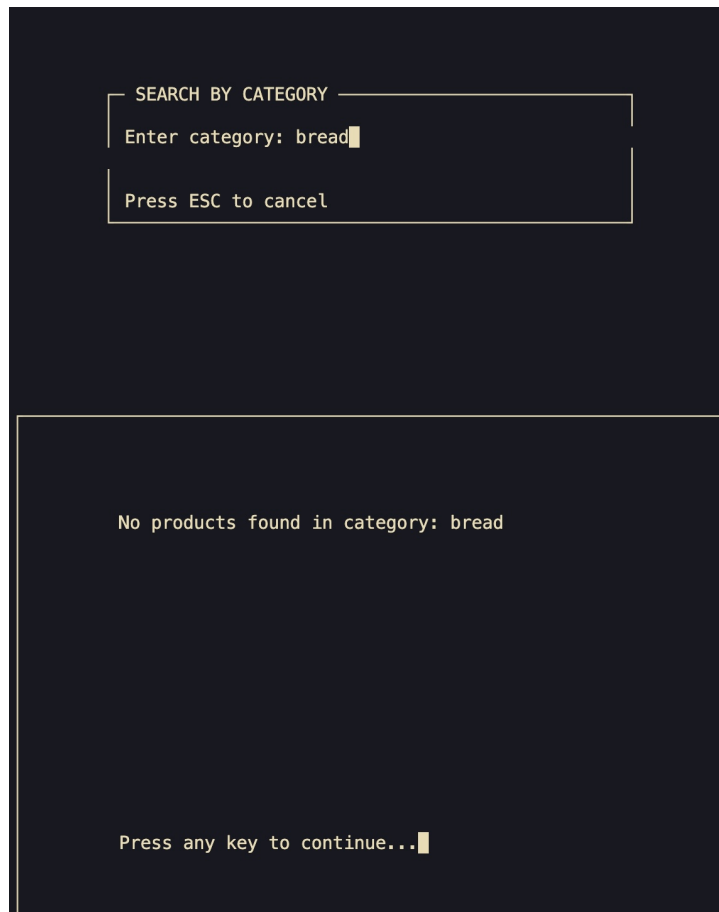


Рис. 14: Поиск по не существующей категории

Весь интерфейс построен на основе оконной модели – каждый экран или диалог представляет собой отдельное окно, что позволяет гибко управлять отображением и обновлением информации. Для реализации были использованы следующие функции ncurses:

- *initscr()* – инициализирует библиотеку ncurses и создает стандартное окно (*stdscr*), представляющее весь терминал. Вызывается в начале каждой функции, работающей с интерфейсом.
- *newwin(height, width, start_y, start_x)* – создает новое окно заданного размера в указанной позиции. Используется для создания модальных диалогов и меню.

- *delwin(win)* – удаляет созданное окно и освобождает память.
- *box(win, 0, 0)* – рисует рамку вокруг окна с использованием символов по умолчанию (ASCII-графика).
- *mvwprintw(win, y, x, format, ...)* – используется для позиционированного вывода форматированной строки.
- *endwin()* – завершает работу с ncurses и восстанавливает терминал в исходное состояние. Вызывается перед выходом из любой функции интерфейса.
- *clear()* – очищает стандартное окно, удаляя весь предыдущий вывод.
- *refresh()* – обновляет физический экран, синхронизируя буфер ncurses с реальным терминалом.

Приведу пример создания главного дисплея.

```
WINDOW *main_win = newwin(17, 55, (rows - 17) / 2, (cols - 55) / 2);
box(main_win, 0, 0);
mvwprintw(main_win, 0, 2, " PRODUCT DATABASE MANAGEMENT ");
```

Для ввода данных используются функции с поддержкой редактирования:

- *wgetch(win)* – получает один символ из окна (неблокирующий ввод с обработкой специальных клавиш).
- *wgetnstr(win, str, size)* – считывает строку ввода с поддержкой редактирования (Backspace, ввод).
- *echo()* – включает отображение вводимых символов на экране.
- *noecho()* – отключает отображение вводимых символов (используется для меню).

- *cbreak()* – переводит терминал в режим посимвольного ввода (без буферизации строки).
- *curs_set(0/1)* – управляет видимостью курсора (0 – скрыт, 1 – видим).
- *wmove(win, y, x)* – перемещает курсор в указанную позицию в окне.
- *wclrtoeol(win)* – очищает строку от текущей позиции курсора до конца.

Во время ввода текста от пользователя каждый введенный символ отображается сразу. Для цены и количества товара ввод специальных символов запрещен (кроме точки в поле для цены), при попытке ввести запрещенный символ, программа его удаляет.

После выполнения задания по разработке программы необходимо исследовать разделяемые библиотеки в полученной программе, используя команду `ldd` и построить соответствующее рекурсивное дерево зависимостей.

```
[tashlykova@tashlykova-1:~/task2/build$ ../../tree.sh price_db
├── libcurses.so.6
│   ├── libtinfo.so.6
│   │   ├── libc.so.6
│   │   │   └── ld-linux-aarch64.so.1
│   │   └── ld-linux-aarch64.so.1
│   ├── libc.so.6
│   │   └── ld-linux-aarch64.so.1
│   └── ld-linux-aarch64.so.1
├── libtinfo.so.6
│   ├── libc.so.6
│   │   └── ld-linux-aarch64.so.1
│   └── ld-linux-aarch64.so.1
├── libz.so.1
│   ├── libc.so.6
│   │   └── ld-linux-aarch64.so.1
│   └── ld-linux-aarch64.so.1
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1
```

Рис. 15: Дерево зависимостей

Были выявлены следующие динамические зависимости:

- `libncurses.so.6` – основная библиотека для создания псевдографического интерфейса. Предоставляет все необходимые функции для работы с окнами, вводом и выводом в терминале.
- `libtinfo.so.6` – библиотека терминальной информации (`terminfo`). Обеспечивает корректную работу с различными типами терминалов. Загружается как зависимость `libncurses`.
- `libz.so.1` – библиотека сжатия данных `zlib`. Используется в программе для вычисления контрольной суммы CRC-32 заголовка базы данных.
- `libc.so.6` – стандартная библиотека языка Си (`GNU C Library`). Предоставляет базовые функции для работы с памятью, файлами, строками и математическими операциями.
- `ld-linux-aarch64.so.1` – динамический загрузчик. Отвечает за загрузку и связывание всех динамических библиотек при запуске программы. Является обязательной зависимостью для всех динамически слинкованных программ в Linux.

3 Результаты работы и эксперименты

Проведем тестирование времени и корректности выполнения функций *palindrome* и *palindrome_recursive* на собственных и случайно сгенерированных данных.

```
tashlykova@tashlykova-1:~/task1/build$ ./bin/test_palindrome 1000000 10000 iter
```

```
tashlykova@tashlykova-1:~/task1/build$ ./bin/test_palindrome 1000000 10000 recursive
```

```
tashlykova@tashlykova-1:~/task1/build$ ./bin/test_palindrome_dynamic 1000000 10000 iter
```

```
tashlykova@tashlykova-1:~/task1/build$ ./bin/test_palindrome_dynamic 1000000 10000 recursive
```

```
tashlykova@tashlykova-1:~/task1/build$ ./bin/test_dlopen 1000000 10000
```

Для генерируемых данных количество итераций: 1000000, длина слова: 10000. Используются 3 типа линковки: статическая, динамическая, динамическая загрузка (dlopen).

Таблица. 2: Тестирование корректности функций

Слово	Длина	Ожидаемый результат	Фактический результат
level...level	135	true	true
hello...hello	55	false	false
bebe...eb	113	true	true
aAA...AAa	112	true	true
friends...friends	126	false	false
A	1	true	true
aA	2	false	false
(пустая строка)	0	true	true

Таким образом, ожидаемый результат совпадает с фактическим результатом, значит функции работают корректно.

Таблица. 3: Тестирование скорости работы функций

Тип линковки	Метод	Время выполнения (с)	Среднее время на слово (мс)
Статическая линковка	итеративный	45.857222	0.045857
Статическая линковка	рекурсивный	46.009406	0.046009
Динамическая линковка	итеративный	46.192764	0.046193
Динамическая линковка	рекурсивный	45.730578	0.045731
Динамическая загрузка через dlopen	итеративный	46.294545	0.046295
Динамическая загрузка через dlopen	рекурсивный	46.695704	0.046696

Таким образом при статической линковке рекурсивный метод медленнее на 0.33%, при динамической – рекурсивный метод быстрее на 1.0%, при динамической загрузке с помощью dlopen – рекурсивный метод медленнее на 0.86%. Можно считать разницу незначительной (дополнительные флаги для оптимизации не используются).

Следующим проведем тестирование с разными флагами компиляции (-O0, -O1, -O2, -Os).

`./scripts/analyze_libs.sh`

`./scripts/test_optimization.sh`

Таблица. 4: Время выполнения программы с различными флагами оптимизации

Флаг	Итеративный (стат.), с.	Рекурсивный (стат.), с.	Итеративный (дин.), с.	Рекурсивный (дин.), с.	dlopen (итер.), с.	dlopen (рек.), с.
O0	4.574	4.612	4.610	4.627	4.638	4.650
O1	4.106	4.119	4.130	4.174	4.126	4.140
O2	3.923	3.896	3.900	3.923	3.896	3.895
Os	4.641	4.626	4.640	4.643	4.627	4.631

Проведем анализ полученных результатов. Флаг -O0 дает самое медленное выполнение программы, при таком подходе код компилируется без каких-либо оптимизаций, все переменные хранятся в памяти.

Флаг -O1 обеспечивает хороший баланс между временем компиляции и производительностью, время выполнения программы немного уменьшается. С таким подходом идет устранение мертвого кода, присутствует оптимизация переходов, имеется начальное распределение переменных по регистрам.

Флаг -O2 дал наилучшую производительность по времени, благодаря развертыванию циклов, оптимизации хвостовой рекурсии, векторизации операций.

Флаг -Os уменьшает размер исполняемого кода, использует более короткие инструкции. Время выполнения программы, так же как и с -O0, медленное.

Далее проведем анализ размеров библиотек, учитывая флаги компиляции.

Таблица. 5: Размеры библиотек

Флаг	Размер <i>libpalindrome.a</i> , К
-O0	2.1
-O1	1.9
-O2	2.0
-Os	1.8

Наименьший размер занимает библиотека при компиляции с флагом -Os, больше всего – с -O0. При этом все исполняемые файлы имеют одинаковый размер (70К) независимо от флага оптимизации (размеры файлов автоматически сохраняются в папку results/).

Опираясь на таблицы 4 и 5, можно сделать следующие выводы:

1. O2 – лучший выбор для максимальной производительности;
2. Os – предпочтителен для систем с ограниченным дисковым пространством;
3. O1 – хороший компромисс для разработки;
4. O0 – используется только для отладки.

Далее проверим работоспособность программы с псевдографическим интерфейсом, для этого будем вводить некорректные данные.

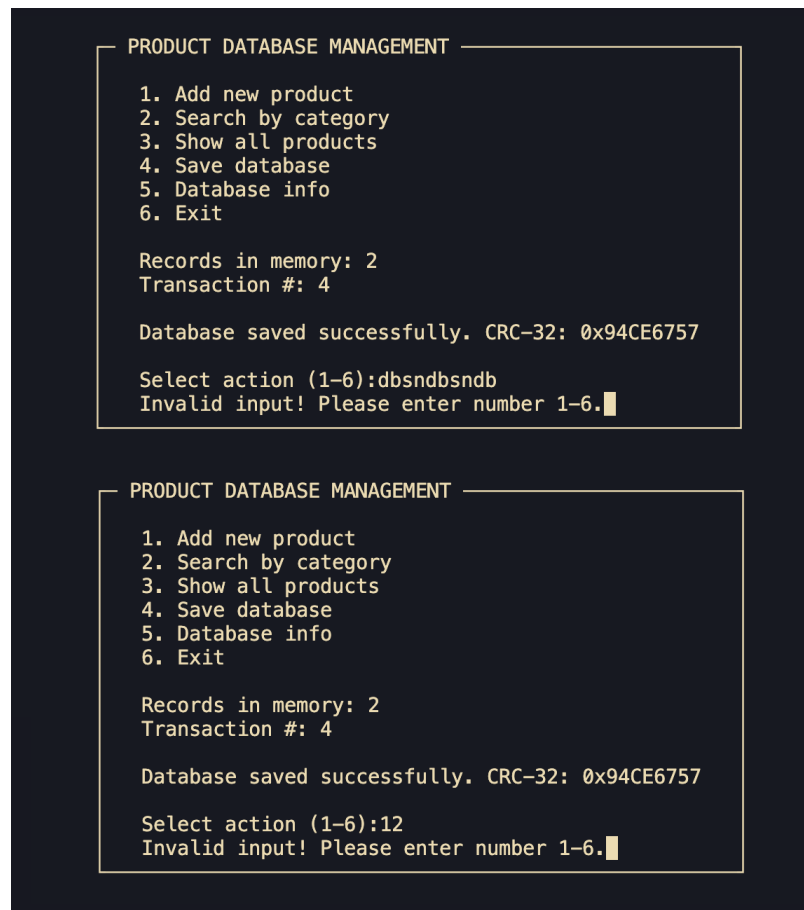


Рис. 16: Ввод некорректных данных при создании нового объекта

Как можно увидеть, в программе предусмотрена защита от ввода некорректных значений, таких как : ввод пустого поля вместо категории или названия продукта, ввод не числовых значений, значений меньше 0 в поля стоимости и количества товара. Для удобства внизу окна выводится причина ошибки. Стоит

отметить, что в поле с ценой пользователь может вводить только цифры, буквы и точку, в поле с количеством – только цифры и буквы. Другие специальные символы автоматически убираются программой.

Также в программе реализована защита от ввода некорректных данных в основном меню.



```
PRODUCT DATABASE MANAGEMENT
1. Add new product
2. Search by category
3. Show all products
4. Save database
5. Database info
6. Exit

Records in memory: 2
Transaction #: 4

Database saved successfully. CRC-32: 0x94CE6757

Select action (1-6):dbsndbsndb
Invalid input! Please enter number 1-6.

PRODUCT DATABASE MANAGEMENT
1. Add new product
2. Search by category
3. Show all products
4. Save database
5. Database info
6. Exit

Records in memory: 2
Transaction #: 4

Database saved successfully. CRC-32: 0x94CE6757

Select action (1-6):12
Invalid input! Please enter number 1-6.
```

Рис. 17: Ввод некорректных данных при выборе команды из основного меню

При вводе неподходящих значений после нажатия ENTER программа пишет сообщение об ошибке, ждет 1 секунду (*parms(1000);*) и очищает поле для ввода.

Заключение

В ходе выполнения летней научно-исследовательской практики были полностью реализованы все поставленные задачи:

1. **Установка и настройка среды виртуализации:** установлен Oracle VirtualBox, развернута виртуальная машина с Debian 12 (логин — `tashlykova`, hostname — `x7k9m2p4`), настроен проброс 22-го порта для SSH-подключения.
2. **Освоение работы в командном интерпретаторе:** изучены базовые команды Linux (`whoami`, `date`, `cal`, `uptime`, `df`, `cp`, `mv`, `touch`, `rm`, `stat`, `file`, `find`, `whereis`, `cat`, `hexdump`, `less`, `grep`, `time`), освоена работа с историей команд, автодополнением и редактированием командной строки, настроена беспарольная SSH-аутентификация по ключам.
3. **Разработка статической и динамической библиотек:** реализована функция проверки палиндрома в двух вариантах (итеративном и рекурсивном) без использования библиотечных функций; созданы статическая (`libpalindrome.a`) и динамическая (`libpalindrome.so`) библиотеки с использованием CMake; реализована динамическая загрузка библиотеки через `dlopen()`.
4. **Тестирование производительности:** проведено тестирование скорости работы функций при различных флагах оптимизации компилятора (`-O0`, `-O1`, `-O2`, `-Os`). Установлено, что флаг `-O2` обеспечивает максимальную производительность (ускорение до 16% по сравнению с `-O0`), а `-Os` дает наименьший размер исполняемого кода. Разница между статической и динамической линковкой составила менее 1%.
5. **Исследование разделяемых библиотек:** с помощью команды `ldd` и

разработанного скрипта `tree.sh` построены рекурсивные деревья зависимостей для программ из задания. Выявлено, что все программы зависят от стандартной библиотеки `libc.so.6` и динамического загрузчика `ld-linux-aarch64.so.1`. Программа с `dlopen` не показывает зависимость от `libpalindrome.so` в выводе `ldd`, так как библиотека загружается во время выполнения.

6. **Разработка программы с псевдографическим интерфейсом:** создана структура `PRICE` (категория, название, цена, количество) и заголовок базы данных с сигнатурой, номером транзакции, количеством записей и контрольной суммой CRC-32. Разработана программа на `ncurses` с главным меню, позволяющая добавлять товары, выполнять поиск по категории с вычислением средней цены товара с максимальным количеством, просматривать все записи, сохранять базу данных и просматривать информацию о базе. Реализована защита от ввода некорректных данных.

В ходе работы были изучены и освоены следующие технологии и навыки:

- Работа с виртуальными машинами и настройка сетевого доступа (проброс портов);
- Командная строка Linux и основные утилиты;
- Системы сборки CMake и make;
- Создание и использование статических и динамических библиотек;
- Загрузка библиотек во время выполнения (`dlopen`, `dlsym`);
- Исследование зависимостей программ с помощью `ldd`;
- Работа с двоичными файлами и контрольными суммами CRC-32;

- Создание псевдографических интерфейсов с библиотекой **ncurses**;
- Оптимизация кода с использованием флагов компилятора;
- Документирование результатов экспериментов.

Средства разработки и инструментарий:

- ОС: GNU/Linux Debian 12 (гостевая), Ubuntu (хостовая);
- Компилятор: GCC 12.2.0;
- Система сборки: CMake 3.25+ и Make;
- Отладка и разработка: Nano, Visual Studio Code;
- Библиотеки: ncurses, zlib;
- Скрипты автоматизации: Bash.

Все задачи, сформулированные в постановке, выполнены в полном объеме. Результаты экспериментов воспроизводимы: разработанные скрипты `test_optimization.sh` и `analyze_libs.sh` позволяют повторить все измерения на системе с аналогичным окружением. Программный код соответствует требованиям индивидуального варианта №14.

Список используемых источников

- [1] Oracle Corporation. Oracle VirtualBox [Электронный ресурс] : официальный сайт. — URL: <https://www.oracle.com/virtualization/virtualbox/> (дата обращения: 18.06.2026).
- [2] КНИТУ-КАИ. МДК.01.04 Системное программирование. Лекция 21. Статические и динамические библиотеки [Электронный ресурс] : учебные материалы. — URL: <https://moodle.kstu.ru/course/section.php?id=69491> (дата обращения: 20.06.2026).
- [3] КНИТУ-КАИ. МДК.01.04 Системное программирование. Лекция 22. Подключение библиотеки во время выполнения: dlopen, dlsym, dlclose. Собственная библиотека [Электронный ресурс] : учебные материалы. — URL: <https://moodle.kstu.ru/course/section.php?id=69492> (дата обращения: 20.06.2026).
- [4] metanit.com. Язык программирования Си. Работа с файлами [Электронный ресурс] : учебный материал. — URL: <https://metanit.com/c/tutorial/10.1.php> (дата обращения: 21.06.2026).
- [5] Gookin D. Programming the Terminal with ncurses : [Электронный ресурс] / D. Gookin. — San Francisco : No Starch Press, 2007. — 576 p. — ISBN 978-1-59327-090-0.

Приложение А. Исходный код файлов задания №1

Листинг 1: Файл src/palindrome.h

```
1 #ifndef PALINDROME_H
2 #define PALINDROME_H
3
4 #include <stdbool.h>
5
6 bool palindrome(char *string, int n);
7 bool palindrome_recursive(char *string, int n);
8 int string_len(char *string);
9
10 #endif
```

Листинг 2: Файл src/palindrome.c

```
1 #include <stdbool.h> // false, true
2
3 bool palindrome(char *string, int n)
4 {
5     for (int i = 0; i < (int)(n / 2); i++)
6     {
7         if (string[i] != string[n - 1 - i])
8             return false;
9     }
10    return true;
11 }
12
13 bool palindrome_recursive(char *string, int n)
14 {
15     if (n == 0 || n == 1)
16         return true;
17     if (string[0] == string[n - 1])
18         return palindrome_recursive(string + 1, n - 2);
19     return false;
20 }
```

```

21
22 int string_len(char *string)
23 {
24     int n = 0;
25     char c = string[0];
26     while (c != '\0')
27         c = string[++n];
28     return n;
29 }

```

Листинг 3: Файл src/CmakeLists.txt

```

1  set(LIB_SOURCES palindrome.c)
2  set(LIB_HEADERS palindrome.h)
3
4  # статическая библиотека
5  if(BUILD_STATIC_LIB)
6      add_library(palindrome_static STATIC ${LIB_SOURCES})
7      target_include_directories(palindrome_static PUBLIC ${
CMAKE_CURRENT_SOURCE_DIR})
8      set_target_properties(palindrome_static PROPERTIES
9          OUTPUT_NAME palindrome
10         CLEAN_DIRECT_OUTPUT 1
11     )
12     install(TARGETS palindrome_static
13         ARCHIVE DESTINATION lib
14     )
15 endif()
16
17 # динамическая библиотека
18 if(BUILD_SHARED_LIB)
19     add_library(palindrome_shared SHARED ${LIB_SOURCES})
20     target_include_directories(palindrome_shared PUBLIC ${
CMAKE_CURRENT_SOURCE_DIR})
21     set_target_properties(palindrome_shared PROPERTIES
22         OUTPUT_NAME palindrome
23         CLEAN_DIRECT_OUTPUT 1
24         VERSION ${PROJECT_VERSION}

```

```

25     SOVERSION 1
26 )
27
28 if(APPLE)
29     set_target_properties(palindrome_shared PROPERTIES
30         INSTALL_NAME_DIR "@rpath"
31         BUILD_WITH_INSTALL_RPATH ON
32     )
33
34     target_link_options(palindrome_shared PRIVATE
35         -Wl,-install_name,@rpath/libpalindrome.dylib
36     )
37 elseif(UNIX)
38     target_link_options(palindrome_shared PRIVATE
39         -Wl,-soname,libpalindrome.so.1
40     )
41 endif()
42
43 install(TARGETS palindrome_shared
44     LIBRARY DESTINATION lib
45     RUNTIME DESTINATION bin
46 )
47 endif()
48
49 install(FILES ${LIB_HEADERS} DESTINATION include)

```

Листинг 4: Файл tests/test_palindrome.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <time.h>
5 #include "palindrome.h"
6
7 #define TEST_WORDS 1000000
8
9 void generate_random_string(char *str, int len)
10 {

```



```

11     for (int i = 0; i < len - 1; i++)
12         str[i] = 'a' + rand() % 26;
13     str[len - 1] = '\\0';
14 }
15
16 double test_speed(int iterations, int word_len, bool use_recursive)
17 {
18     char *word = malloc(word_len);
19     clock_t start, end;
20     double cpu_time_used;
21     bool result;
22
23     start = clock();
24
25     for (int i = 0; i < iterations; i++)
26     {
27         generate_random_string(word, word_len);
28         int len = string_len(word);
29
30         if (use_recursive)
31             result = palindrome_recursive(word, len);
32         else
33             result = palindrome(word, len);
34         (void)result;
35     }
36
37     end = clock();
38     cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
39
40     free(word);
41     return cpu_time_used;
42 }
43
44 int main(int argc, char *argv[])
45 {
46     int iterations = TEST_WORDS;
47     int word_len = 10;
48     bool use_recursive = false;

```



```

79     int len = string_len(test_words[i]);
80     bool result1 = palindrome(test_words[i], len);
81     bool result2 = palindrome_recursive(test_words[i], len);
82     printf("'s' (len=%d): итеративный=%d, рекурсивный=%d\n", test_words[i],
len, result1, result2);
83 }
84
85     return 0;
86 }

```

Листинг 5: Файл tests/test_dlopen.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <time.h>
5
6  #include <dlfcn.h>
7  #include <stdbool.h>
8  #include <unistd.h>
9  #include <libgen.h>
10
11 #define TEST_WORDS 1000000
12
13 typedef bool (*palindrome_func_t)(char *, int);
14 typedef int (*string_len_func_t)(char *);
15
16 char* get_library_path() {
17     static char lib_path[512];
18     char *lib_dir = getenv("PALINDROME_LIB_DIR");
19     if (lib_dir != NULL) {
20         #ifdef __APPLE__
21             snprintf(lib_path, sizeof(lib_path), "%s/libpalindrome.dylib",
lib_dir);
22         #else
23             snprintf(lib_path, sizeof(lib_path), "%s/libpalindrome.so", lib_dir);
24         #endif
25         return lib_path;

```

```

26     }
27
28     #ifdef __APPLE__
29     if (access("./libpalindrome.dylib", F_OK) == 0) {
30         snprintf(lib_path, sizeof(lib_path), "./libpalindrome.dylib");
31         return lib_path;
32     }
33     #else
34     if (access("./libpalindrome.so", F_OK) == 0) {
35         snprintf(lib_path, sizeof(lib_path), "./libpalindrome.so");
36         return lib_path;
37     }
38     #endif
39
40     char exe_path[512];
41     ssize_t len = readlink("/proc/self/exe", exe_path, sizeof(exe_path) - 1);
42     if (len != -1) {
43         exe_path[len] = '\0';
44         char *dir = dirname(exe_path);
45         #ifdef __APPLE__
46         snprintf(lib_path, sizeof(lib_path), "%s/../../lib/libpalindrome.dylib",
47 dir);
48         if (access(lib_path, F_OK) == 0) {
49             return lib_path;
50         }
51         snprintf(lib_path, sizeof(lib_path), "%s/libpalindrome.dylib", dir);
52         #else
53         snprintf(lib_path, sizeof(lib_path), "%s/../../lib/libpalindrome.so",
54 dir);
55         if (access(lib_path, F_OK) == 0) {
56             return lib_path;
57         }
58         snprintf(lib_path, sizeof(lib_path), "%s/libpalindrome.so", dir);
59         #endif
60         return lib_path;
61     }
62
63     #ifdef __APPLE__

```

```

62     snprintf(lib_path, sizeof(lib_path), "libpalindrome.dylib");
63     #else
64     snprintf(lib_path, sizeof(lib_path), "libpalindrome.so");
65     #endif
66     return lib_path;
67 }
68
69 void generate_random_string(char *str, int len) {
70     for (int i = 0; i < len - 1; i++) {
71         str[i] = 'a' + rand() % 26;
72     }
73     str[len - 1] = '\0';
74 }
75
76 double test_speed(int iterations, int word_len, palindrome_func_t
    palindrome_func, string_len_func_t len_func) {
77     char *word = malloc(word_len);
78     clock_t start, end;
79     double cpu_time_used;
80     bool result;
81
82     start = clock();
83
84     for (int i = 0; i < iterations; i++) {
85         generate_random_string(word, word_len);
86         int len = len_func(word);
87         result = palindrome_func(word, len);
88         (void)result;
89     }
90
91     end = clock();
92     cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
93
94     free(word);
95     return cpu_time_used;
96 }
97
98 int main(int argc, char *argv[]) {

```

```

99     void *handle;
100     palindrome_func_t palindrome_iter, palindrome_rec;
101     string_len_func_t string_len_func;
102     char *error;
103
104     char *lib_path = get_library_path();
105     printf("Попытка загрузить библиотеку: %s\n", lib_path);
106
107     handle = dlopen(lib_path, RTLD_LAZY);
108     if (!handle) {
109         fprintf(stderr, "Ошибка загрузки библиотеки: %s\n", dlerror());
110         return 1;
111     }
112
113     printf("Библиотека успешно загружена\n");
114
115     // указатели на функции
116     palindrome_iter = (palindrome_func_t)dlsym(handle, "palindrome");
117     if ((error = dlerror()) != NULL) {
118         fprintf(stderr, "Ошибка получения функции palindrome: %s\n", error);
119         dlclose(handle);
120         return 1;
121     }
122
123     palindrome_rec = (palindrome_func_t)dlsym(handle, "palindrome_recursive")
124 ;
125     if ((error = dlerror()) != NULL) {
126         fprintf(stderr, "Ошибка получения функции palindrome_recursive: %s\n",
127 error);
128         dlclose(handle);
129         return 1;
130     }
131
132     string_len_func = (string_len_func_t)dlsym(handle, "string_len");
133     if ((error = dlerror()) != NULL) {
134         fprintf(stderr, "Ошибка получения функции string_len: %s\n", error);
135         dlclose(handle);
136         return 1;

```



```

friendsfriendsfriendsfriendsfriendsfriendsfriendsfriendsfriendsfriendsfriends
", "", "A", "aA"};
165     int num_tests = 8;
166
167     for (int i = 0; i < num_tests; i++) {
168         int len = string_len_func(test_words[i]);
169         bool result1 = palindrome_iter(test_words[i], len);
170         bool result2 = palindrome_rec(test_words[i], len);
171         printf("'s' (len=%d): итеративный=%d, рекурсивный=%d\n",
172             test_words[i], len, result1, result2);
173     }
174
175     dlclose(handle);
176     return 0;
177 }

```

Листинг 6: Файл tests/CmakeLists.txt

```

1 # тест со статической линковкой
2 add_executable(test_palindrome test_palindrome.c)
3
4 # определение макроса STATIC_BUILD для статической версии
5 target_compile_definitions(test_palindrome PRIVATE STATIC_BUILD)
6
7 target_link_libraries(test_palindrome palindrome_static)
8 target_include_directories(test_palindrome PRIVATE ${CMAKE_SOURCE_DIR}/src)
9
10 add_test(NAME test_palindrome_static COMMAND test_palindrome 1000 10 iter)
11
12 # тест с динамической линковкой
13 add_executable(test_palindrome_dynamic test_palindrome.c)
14
15 # НЕ определяем STATIC_BUILD для динамической версии
16 target_link_libraries(test_palindrome_dynamic palindrome_shared)
17 target_include_directories(test_palindrome_dynamic PRIVATE ${CMAKE_SOURCE_DIR}
    }/src)
18
19

```



```

20 if(APPLE)
21     set_target_properties(test_palindrome_dynamic PROPERTIES
22         BUILD_RPATH "${CMAKE_LIBRARY_OUTPUT_DIRECTORY}"
23         INSTALL_RPATH "${CMAKE_INSTALL_PREFIX}/lib"
24     )
25 else()
26     set_target_properties(test_palindrome_dynamic PROPERTIES
27         BUILD_RPATH "${CMAKE_LIBRARY_OUTPUT_DIRECTORY}"
28         INSTALL_RPATH "${CMAKE_INSTALL_PREFIX}/lib"
29     )
30 endif()

```

Листинг 7: Файл scripts/test_optimization.sh

```

1 echo "Тестирование оптимизаций компилятора с CMake :"
2 echo ""
3
4 # флаги оптимизации
5 OPTIMIZATIONS=("O0" "O1" "O2" "Os")
6 TEST_ITERATIONS=100000000
7 WORD_LENGTH=100000000
8
9 if [[ "$OSTYPE" == "darwin"* ]]; then
10     # macOS
11     NPROC=$(sysctl -n hw.ncpu)
12 else
13     # Linux
14     NPROC=$(nproc 2>/dev/null || echo 4)
15 fi
16
17 echo "Используется $NPROC ядер для сборки"
18 echo ""
19
20 # директория для результатов
21 mkdir -p results
22
23 for opt in "${OPTIMIZATIONS[@]}"; do
24     echo "Флаг оптимизации: -$opt"

```

```

25 echo "-----"
26
27 # директория для сборки
28 BUILD_DIR="build_${opt}"
29 mkdir -p $BUILD_DIR
30 cd $BUILD_DIR
31
32 # конфигурация с флагом оптимизации
33 cmake .. -DCMAKE_BUILD_TYPE=Release -DCMAKE_C_FLAGS_RELEASE="-${opt}"
34
35 # сборка
36 make clean > /dev/null 2>&1
37 make -j$NPROC > /dev/null 2>&1
38
39 # Проверяем наличие библиотеки и копируем для dlopen
40 if [ -f "lib/libpalindrome.dylib" ]; then
41     cp lib/libpalindrome.dylib bin/
42     cd bin
43     ln -sf libpalindrome.dylib libpalindrome.so 2>/dev/null
44     cd ..
45 fi
46
47 echo "Статическая линковка итеративный(): "
48 time ./bin/test_palindrome $TEST_ITERATIONS $WORD_LENGTH iter
49
50 echo ""
51 echo "Статическая линковка рекурсивный(): "
52 time ./bin/test_palindrome $TEST_ITERATIONS $WORD_LENGTH recursive
53
54 echo ""
55 echo "Динамическая линковка итеративный(): "
56 time ./bin/test_palindrome_dynamic $TEST_ITERATIONS $WORD_LENGTH iter
57
58 echo ""
59 echo "Динамическая линковка рекурсивный(): "
60 time ./bin/test_palindrome_dynamic $TEST_ITERATIONS $WORD_LENGTH
recursive
61

```

```

62     echo ""
63     echo "dlopen итеративный():"
64     export PALINDROME_LIB_DIR=$(pwd)/lib
65     time ./bin/test_dlopen $TEST_ITERATIONS $WORD_LENGTH
66
67     echo ""
68     # сохраняем размеры файлов
69     echo "Размеры файлов:" >> ../results/sizes_${opt}.txt
70     ls -lh bin/test_palindrome bin/test_palindrome_dynamic bin/test_dlopen
71     lib/*.a 2>/dev/null >> ../results/sizes_${opt}.txt
72     if [ -f "lib/libpalindrome.dylib" ]; then
73         ls -lh lib/libpalindrome.dylib >> ../results/sizes_${opt}.txt
74     fi
75
76     cd ..
77     echo ""
78
79     done
80
81     echo Сравнение " размеров исполняемых файлов:"
82     echo ""
83     for opt in "${OPTIMIZATIONS[@]"; do
84         echo Оптимизация " -${opt}:"
85         cat results/sizes_${opt}.txt
86         echo ""
87     done

```

Листинг 8: Файл scripts/analyze_libs.sh

```

1  echo "Анализ разделяемых библиотек (CMake) :"
2  echo ""
3
4  # сборка в режиме release
5  mkdir -p build_analysis
6  cd build_analysis
7  cmake .. -DCMAKE_BUILD_TYPE=Release
8  make -j$(nproc) > /dev/null 2>&1
9
10 echo "1. Статическая линковка:"

```

```

11 ldd bin/test_palindrome
12 echo ""
13
14 echo "2. Динамическая линковка:"
15 ldd bin/test_palindrome_dynamic
16 echo ""
17
18 echo "3. Программа с dlopen:"
19 ldd bin/test_dlopen
20 echo ""
21
22 echo "4. Зависимости динамической библиотеки:"
23 ldd lib/libpalindrome.so
24 echo ""
25
26 echo "5. Размеры файлов:"
27 ls -lh bin/test_palindrome bin/test_palindrome_dynamic bin/test_dlopen lib/*.
    a lib/*.so
28 echo ""
29
30 cd ..
31 cd ..

```

Листинг 9: Файл CmakeLists.txt

```

1 cmake_minimum_required(VERSION 4.0)
2 project(PalindromeProject VERSION 1.0 LANGUAGES C)
3
4 set(CMAKE_C_STANDARD 11)
5 set(CMAKE_C_STANDARD_REQUIRED ON)
6
7 option(BUILD_STATIC_LIB "Build static library" ON)
8 option(BUILD_SHARED_LIB "Build shared library" ON)
9 option(BUILD_TESTS "Build test executables" ON)
10 option(BUILD_DLOPEN_TEST "Build dlopen test" ON)
11
12 set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
13 set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)

```

```
14 set(CMAKE_ARCHIVE_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)
15
16 add_subdirectory(src)
17 if(BUILD_TESTS)
18     add_subdirectory(tests)
19 endif()
20
21 enable_testing()
22
23 message(STATUS "Project: ${PROJECT_NAME}")
24 message(STATUS "Version: ${PROJECT_VERSION}")
25 message(STATUS "Build type: ${CMAKE_BUILD_TYPE}")
26 message(STATUS "C Compiler: ${CMAKE_C_COMPILER}")
27 message(STATUS "Platform: ${CMAKE_SYSTEM_NAME}")
```

Приложение В. Исходный код файлов задания №2

Листинг 10: Файл src/price.h

```
1 #ifndef PRICE_H
2 #define PRICE_H
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <stdbool.h>
8 #include <zlib.h>      // CRC-32
9
10
11 typedef struct
12 {
13     char *category;
14     char *good;
15     double price;
16     unsigned int num;
17 } PRICE;
18
19 typedef struct
20 {
21     char signature[4];
22     unsigned int tr_num;
23     unsigned int st_count;
24     unsigned int crc32;
25 } DB_HEADER;
26
27 DB_HEADER create_header(const char *signature, unsigned int tx_num, unsigned
    int count, const PRICE *data);
28 unsigned int get_crc32(const PRICE *data, unsigned int count);
29 void save_database(const char *filename, DB_HEADER *header, PRICE *data,
    unsigned int count);
30 DB_HEADER load_database(const char *filename, PRICE **data);
```

```

31 void free_price(PRICE *item);
32
33
34 #endif

```

Листинг 11: Файл src/price.c

```

1  #include "price.h"
2  #include <stdio.h>
3  #include <zlib.h>
4  #include <stdbool.h> // false, true
5
6  unsigned int get_crc32(const PRICE *data, unsigned int count)
7  {
8      uLong crc = crc32(0L, Z_NULL, 0);
9
10     for (unsigned int i = 0; i < count; i++)
11     {
12         // category hash
13         crc = crc32(crc, (const Bytef *)data[i].category, strlen(data[i].
category));
14         // product name hash
15         crc = crc32(crc, (const Bytef *)data[i].good, strlen(data[i].good));
16         // price hash
17         crc = crc32(crc, (const Bytef *)&data[i].price, sizeof(double));
18         // quantity hash
19         crc = crc32(crc, (const Bytef *)&data[i].num, sizeof(unsigned int));
20     }
21
22     return (unsigned int)crc;
23 }
24
25 DB_HEADER create_header(const char *signature, unsigned int tr_num, unsigned
int count, const PRICE *data)
26 {
27     DB_HEADER header;
28     strncpy(header.signature, signature, 4);
29     // header.signature[4] = '\0';

```

```

30     header.tr_num = tr_num;
31     header.st_count = count;
32     header.crc32 = get_crc32(data, count);
33
34     return header;
35 }
36
37 void save_database(const char *filename, DB_HEADER *header, PRICE *data,
    unsigned int count)
38 {
39     FILE *file = fopen(filename, "wb");
40     if (!file)
41     {
42         fprintf(stderr, "Error opening file\n");
43         return;
44     }
45     fwrite(header, sizeof(DB_HEADER), 1, file);
46
47     for (unsigned int i = 0; i < count; i++)
48     {
49         unsigned int len = strlen(data[i].category) + 1;
50         fwrite(&len, sizeof(unsigned int), 1, file);
51         fwrite(data[i].category, sizeof(char), len, file);
52
53         len = strlen(data[i].good) + 1;
54         fwrite(&len, sizeof(unsigned int), 1, file);
55         fwrite(data[i].good, sizeof(char), len, file);
56
57         fwrite(&data[i].price, sizeof(double), 1, file);
58         fwrite(&data[i].num, sizeof(unsigned int), 1, file);
59     }
60     fclose(file);
61     printf("Database saved to file: %s\n", filename);
62     printf("Records count: %u\n", count);
63     printf("CRC-32: 0x%08X\n", header->crc32);
64 }
65
66 DB_HEADER load_database(const char *filename, PRICE **data)

```



```

67 {
68     DB_HEADER header = {0};
69     FILE *file = fopen(filename, "rb");
70
71     if (!file)
72     {
73         fprintf(stderr, "Error opening file\n");
74         return header;
75     }
76
77     size_t read = fread(&header, sizeof(DB_HEADER), 1, file);
78     if (read != 1)
79     {
80         fprintf(stderr, "Error reading header\n");
81         fclose(file);
82         return header;
83     }
84
85     *data = (PRICE *)malloc(header.st_count * sizeof(PRICE));
86     if (!*data)
87     {
88         fprintf(stderr, "Memory allocation error\n");
89         fclose(file);
90         return header;
91     }
92
93     for (unsigned int i = 0; i < header.st_count; i++)
94     {
95         unsigned int len;
96         char *str;
97
98         fread(&len, sizeof(unsigned int), 1, file);
99         str = (char *)malloc(len * sizeof(char));
100        fread(str, sizeof(char), len, file);
101        (*data)[i].category = str;
102
103        fread(&len, sizeof(unsigned int), 1, file);
104        str = (char *)malloc(len * sizeof(char));

```

```

105     fread(str, sizeof(char), len, file);
106     (*data)[i].good = str;
107
108     fread(&(*data)[i].price, sizeof(double), 1, file);
109     fread(&(*data)[i].num, sizeof(unsigned int), 1, file);
110 }
111
112 fclose(file);
113 printf("Database loaded from file: %s\n", filename);
114 printf("Records count: %u\n", header.st_count);
115 printf("Transaction number: %u\n", header.tr_num);
116 printf("CRC-32: 0x%08X\n", header.crc32);
117
118 return header;
119 }
120
121 void free_price(PRICE *item)
122 {
123     if (item->category)
124     {
125         free(item->category);
126         item->category = NULL;
127     }
128     if (item->good)
129     {
130         free(item->good);
131         item->good = NULL;
132     }
133 }

```

Листинг 12: Файл src/main.c

```

1 #include <ncurses.h>
2 #include <ctype.h>
3 #include "price.h"
4
5 PRICE *items = NULL;
6 unsigned int item_count = 0;

```

```

7 DB_HEADER header;
8 char filename[256] = "databasetest.bin";
9
10 int input_data_ncurses();
11 int search_category_ncurses();
12 void search_by_category(const char *category);
13 void show_all_products();
14 void show_database_info();
15 void save_with_message(char *signature);
16 void draw_main_menu(WINDOW *win, const char *status_msg);
17 int get_menu_choice(WINDOW *win);
18
19 void draw_main_menu(WINDOW *win, const char *status_msg)
20 {
21     werase(win);
22     box(win, 0, 0);
23     mvwprintw(win, 0, 2, " PRODUCT DATABASE MANAGEMENT ");
24
25     mvwprintw(win, 2, 4, "1. Add new product");
26     mvwprintw(win, 3, 4, "2. Search by category");
27     mvwprintw(win, 4, 4, "3. Show all products");
28     mvwprintw(win, 5, 4, "4. Save database");
29     mvwprintw(win, 6, 4, "5. Database info");
30     mvwprintw(win, 7, 4, "6. Exit");
31
32     mvwprintw(win, 9, 4, "Records in memory: %u", item_count);
33     mvwprintw(win, 10, 4, "Transaction #: %u", header.tr_num);
34
35     if (status_msg && strlen(status_msg) > 0)
36     {
37         mvwprintw(win, 12, 4, "%s CRC-32: 0x%08X", status_msg, header.crc32);
38     }
39     mvwprintw(win, 14, 4, "Select action (1-6): ");
40     wrefresh(win);
41 }
42
43 int get_menu_choice(WINDOW *win)
44 {

```

```

45 char input[256];
46 int choice;
47
48 while (1)
49 {
50     mvwprintw(win, 14, 24, "                ");
51     wmove(win, 14, 24);
52     wrefresh(win);
53
54     echo();
55     wgetnstr(win, input, sizeof(input) - 1);
56     noecho();
57
58     input[strcspn(input, "\n")] = '\0';
59
60     int valid_input = 1;
61     if (input[0] == '\0')
62         valid_input = 0;
63     else
64     {
65         for (int i = 0; input[i] != '\0'; i++)
66         {
67             if (!isdigit(input[i]))
68             {
69                 valid_input = 0;
70                 break;
71             }
72         }
73     }
74
75     if (valid_input)
76     {
77         choice = atoi(input);
78         if (choice >= 1 && choice <= 6)
79             return choice;
80     }
81
82     mvwprintw(win, 15, 4, "Invalid input! Please enter number 1-6.");

```

```

83         wrefresh(win);
84         napms(1000);
85         mvwprintw(win, 15, 4, "                                     ");
86         wrefresh(win);
87     }
88 }
89
90 int input_data_ncurses()
91 {
92     initscr();
93     cbreak();
94     echo();
95     keypad(stdscr, TRUE);
96     clear();
97     refresh();
98
99     int height = 14;
100    int width = 55;
101    int start_y = 2;
102    int start_x = 10;
103
104    WINDOW *win = newwin(height, width, start_y, start_x);
105    box(win, 0, 0);
106    mvwprintw(win, 0, 2, " ENTER PRODUCT DATA ");
107    mvwprintw(win, 13, 2, "Press ESC to cancel");
108    wrefresh(win);
109
110    char category[256] = "";
111    char good[256] = "";
112    char price_str[256] = "";
113    char num_str[256] = "";
114
115    int valid = 0;
116    int cancelled = 0;
117    double price = 0;
118    unsigned int num = 0;
119
120    while (!valid && !cancelled)

```

```

121 {
122     werase(win);
123     box(win, 0, 0);
124     mvwprintw(win, 0, 2, " ENTER PRODUCT DATA ");
125     mvwprintw(win, 13, 2, "Press ESC to cancel");
126     wrefresh(win);
127
128     mvwprintw(win, 2, 2, "Category: ");
129     wmove(win, 2, 14);
130     wclrtoeol(win);
131     wrefresh(win);
132
133     int ch;
134     int pos = 0;
135     while (1)
136     {
137         ch = wgetch(win);
138         if (ch == 27) // esc
139         {
140             cancelled = 1;
141             break;
142         }
143         if (ch == 10) // enter
144         {
145             category[pos] = '\0';
146             break;
147         }
148         if (ch == KEY_BACKSPACE || ch == 127) // backspace or delete
149         {
150             if (pos > 0)
151             {
152                 pos--;
153                 category[pos] = '\0';
154                 mvwprintw(win, 2, 14, "%-30s", category);
155                 wmove(win, 2, 14 + pos);
156                 wrefresh(win);
157             }
158             continue;

```

```

159     }
160     if (pos < 255 && ch >= 32 && ch <= 126) // разрешены только цифры и
точка
161     {
162         category[pos++] = ch;
163         category[pos] = '\0';
164         mvwprintw(win, 2, 14, "%s", category);
165         wrefresh(win);
166     }
167 }
168 if (cancelled)
169     break;
170
171 mvwprintw(win, 4, 2, "Product name: ");
172 wmove(win, 4, 16);
173 wclrtoeol(win);
174 wrefresh(win);
175
176 pos = 0;
177 while (1)
178 {
179     ch = wgetch(win);
180     if (ch == 27)
181     {
182         cancelled = 1;
183         break;
184     }
185     if (ch == 10)
186     {
187         good[pos] = '\0';
188         break;
189     }
190     if (ch == KEY_BACKSPACE || ch == 127)
191     {
192         if (pos > 0)
193         {
194             pos--;
195             good[pos] = '\0';

```

```

196         mvwprintw(win, 4, 16, "%-30s", good);
197         wmove(win, 4, 16 + pos);
198         wrefresh(win);
199     }
200     continue;
201 }
202 if (pos < 255 && ch >= 32 && ch <= 126)
203 {
204     good[pos++] = ch;
205     good[pos] = '\0';
206     mvwprintw(win, 4, 16, "%s", good);
207     wrefresh(win);
208 }
209 }
210 if (cancelled)
211     break;
212
213 mvwprintw(win, 6, 2, "Cost (rub): ");
214 wmove(win, 6, 14);
215 wclrtoeol(win);
216 wrefresh(win);
217
218 pos = 0;
219 while (1)
220 {
221     ch = wgetch(win);
222     if (ch == 27)
223     {
224         cancelled = 1;
225         break;
226     }
227     if (ch == 10)
228     {
229         price_str[pos] = '\0';
230         break;
231     }
232     if (ch == KEY_BACKSPACE || ch == 127)
233     {

```



```

234         if (pos > 0)
235         {
236             pos--;
237             price_str[pos] = '\0';
238             mvwprintw(win, 6, 14, "%-30s", price_str);
239             wmove(win, 6, 14 + pos);
240             wrefresh(win);
241         }
242         continue;
243     }
244     if (pos < 255 && ((ch >= '0' && ch <= '9') || ch == '.'))
245     {
246         price_str[pos++] = ch;
247         price_str[pos] = '\0';
248         mvwprintw(win, 6, 14, "%s", price_str);
249         wrefresh(win);
250     }
251 }
252 if (cancelled)
253     break;
254
255 mvwprintw(win, 8, 2, "Amount: ");
256 wmove(win, 8, 10);
257 wclrtoeol(win);
258 wrefresh(win);
259
260 pos = 0;
261 while (1)
262 {
263     ch = wgetch(win);
264     if (ch == 27)
265     {
266         cancelled = 1;
267         break;
268     }
269     if (ch == 10)
270     {
271         num_str[pos] = '\0';

```

```

272         break;
273     }
274     if (ch == KEY_BACKSPACE || ch == 127)
275     {
276         if (pos > 0)
277         {
278             pos--;
279             num_str[pos] = '\0';
280             mvwprintw(win, 8, 10, "%-30s", num_str);
281             wmove(win, 8, 10 + pos);
282             wrefresh(win);
283         }
284         continue;
285     }
286     if (pos < 255 && ch >= '0' && ch <= '9')
287     {
288         num_str[pos++] = ch;
289         num_str[pos] = '\0';
290         mvwprintw(win, 8, 10, "%s", num_str);
291         wrefresh(win);
292     }
293 }
294 if (cancelled)
295     break;
296
297 // валидация
298 int valid_price = 1;
299 int valid_num = 1;
300 int has_dot = 0;
301
302 if (strlen(category) == 0 || strlen(good) == 0)
303 {
304     valid = 0;
305     mvwprintw(win, 11, 2, "ERROR: Category and Product name cannot be
empty!");
306     wrefresh(win);
307     getch(); // ожидание нажатия клавиши
308     continue;

```

```

309     }
310
311     if (strlen(price_str) == 0)
312     {
313         valid_price = 0;
314     }
315     else
316     {
317         for (int i = 0; price_str[i] != '\0'; i++)
318         {
319             if (price_str[i] == '.')
320             {
321                 if (has_dot)
322                 {
323                     valid_price = 0;
324                     break;
325                 }
326                 has_dot = 1;
327             }
328             else if (!isdigit(price_str[i]))
329             {
330                 valid_price = 0;
331                 break;
332             }
333         }
334     }
335
336     if (strlen(num_str) == 0)
337         valid_num = 0;
338     else
339     {
340         for (int i = 0; num_str[i] != '\0'; i++)
341         {
342             if (!isdigit(num_str[i]))
343             {
344                 valid_num = 0;
345                 break;
346             }

```

```

347         }
348     }
349
350     if (valid_price && valid_num)
351     {
352         price = atof(price_str);
353         num = atoi(num_str);
354
355         if (price <= 0 || num == 0)
356         {
357             valid = 0;
358             if (price <= 0)
359                 mvwprintw(win, 11, 2, "ERROR: Price must be greater than
360 0!");
361
362             else
363                 mvwprintw(win, 11, 2, "ERROR: Amount must be greater than
364 0!");
365
366             wrefresh(win);
367             getch();
368             continue;
369         }
370         valid = 1;
371     }
372     else
373     {
374         valid = 0;
375         if (!valid_price)
376             mvwprintw(win, 11, 2, "ERROR: Price must be a positive number
377 (e.g. 99.50)");
378         else if (!valid_num)
379             mvwprintw(win, 11, 2, "ERROR: Amount must be a positive
380 integer (e.g. 10)");
381
382         wrefresh(win);
383         getch();
384     }
385 }

```

```

381     if (cancelled)
382     {
383         delwin(win);
384         endwin();
385         return 0;
386     }
387
388     items = (PRICE *)realloc(items, (item_count + 1) * sizeof(PRICE));
389     if (items)
390     {
391         items[item_count].category = strdup(category);
392         items[item_count].good = strdup(good);
393         items[item_count].price = price;
394         items[item_count].num = num;
395         item_count++;
396
397         werase(win);
398         box(win, 0, 0);
399         mvwprintw(win, 0, 2, " ENTER PRODUCT DATA ");
400         mvwprintw(win, 5, 2, "Product added successfully!");
401         mvwprintw(win, 7, 2, "Category: %s", category);
402         mvwprintw(win, 8, 2, "Name: %s", good);
403         mvwprintw(win, 9, 2, "Price: %.2f rub.", price);
404         mvwprintw(win, 10, 2, "Amount: %u", num);
405         mvwprintw(win, 12, 2, "Press any key to continue...");
406         wrefresh(win);
407         getch();
408
409         delwin(win);
410         endwin();
411         return 1;
412     }
413
414     delwin(win);
415     endwin();
416     return 0;
417 }
418

```

```

419 int search_category_ncurses()
420 {
421     char category[256] = "";
422     int rows, cols;
423     WINDOW *win;
424     int ch;
425     int pos = 0;
426     int cancelled = 0;
427
428     initscr();
429     cbreak();
430     echo();
431     keypad(stdscr, TRUE);
432     clear();
433     refresh();
434
435     getmaxyx(stdscr, rows, cols);
436
437     win = newwin(7, 50, (rows - 7) / 2, (cols - 50) / 2);
438     box(win, 0, 0);
439     mvwprintw(win, 0, 2, " SEARCH BY CATEGORY ");
440     mvwprintw(win, 2, 2, "Enter category: ");
441     mvwprintw(win, 5, 2, "Press ESC to cancel");
442     wrefresh(win);
443
444     mvwprintw(win, 2, 18, "
445     wmove(win, 2, 18);
446     wrefresh(win);
447
448     while (1)
449     {
450         ch = wgetch(win);
451         if (ch == 27)
452         { // esc
453             cancelled = 1;
454             break;
455         }
456         if (ch == 10)

```

```

457     { // enter
458         category[pos] = '\0';
459         break;
460     }
461     if (ch == KEY_BACKSPACE || ch == 127)
462     {
463         if (pos > 0)
464         {
465             pos--;
466             category[pos] = '\0';
467             mvwprintw(win, 2, 18, "%-30s", category);
468             wmove(win, 2, 18 + pos);
469             wrefresh(win);
470         }
471         continue;
472     }
473     if (pos < 255 && ch >= 32 && ch <= 126)
474     {
475         category[pos++] = ch;
476         category[pos] = '\0';
477         mvwprintw(win, 2, 18, "%s", category);
478         wrefresh(win);
479     }
480 }
481
482 delwin(win);
483 endwin();
484
485 if (cancelled || strlen(category) == 0)
486 {
487     return 0; // отмена
488 }
489
490 search_by_category(category);
491 return 1; // успешно
492 }
493
494 void search_by_category(const char *category)

```

```

495 {
496     bool found = false;
497     unsigned int max_num = 0;
498     unsigned int count_found = 0;
499     double total_price = 0.0;
500     int indices[100] = {0};
501
502     for (unsigned int i = 0; i < item_count; i++)
503     {
504         if (strcmp(items[i].category, category) == 0)
505         {
506             found = true;
507             if (items[i].num > max_num)
508             {
509                 max_num = items[i].num;
510                 count_found = 1;
511                 total_price = items[i].price;
512                 indices[0] = i;
513             }
514             else if (items[i].num == max_num)
515             {
516                 indices[count_found++] = i;
517                 total_price += items[i].price;
518             }
519         }
520     }
521
522     initscr();
523     clear();
524     refresh();
525     box(stdscr, 0, 0);
526
527     if (!found)
528     {
529         mvprintw(5, 10, "No products found in category: %s", category);
530     }
531     else
532     {

```



```

533     mvprintw(2, 10, "SEARCH RESULTS ");
534     mvprintw(3, 10, "Category: %s", category);
535     mvprintw(5, 10, "Maximum quantity: %u", max_num);
536     mvprintw(6, 10, "Number of products with max num: %u", count_found);
537     mvprintw(7, 10, "Average price: %.2f rub.", total_price / count_found
);
538
539     mvprintw(9, 10, "Products with max quantity:");
540     for (unsigned int i = 0; i < count_found; i++)
541     {
542         int idx = indices[i];
543         mvprintw(11 + i, 15, "%s - %.2f rub.", items[idx].good, items[idx
].price);
544     }
545 }
546
547 mvprintw(20, 10, "Press any key to continue...");
548 refresh();
549 getch();
550 endwin();
551 }
552
553 void show_all_products()
554 {
555     initscr();
556     clear();
557     refresh();
558
559     int rows, cols;
560     getmaxyx(stdscr, rows, cols);
561
562     if (item_count == 0)
563     {
564         mvprintw(rows / 2, (cols - 20) / 2, "Database is empty!");
565         getch();
566         endwin();
567         return;
568     }

```

```

569
570     box(stdscr, 0, 0);
571     mvprintw(0, 2, " ALL PRODUCTS ");
572
573     int line = 2;
574     for (unsigned int i = 0; i < item_count && line < rows - 2; i++)
575     {
576         mvprintw(line++, 2, "[%u] Category: %s", i + 1, items[i].category);
577         mvprintw(line++, 4, "Name: %s", items[i].good);
578         mvprintw(line++, 4, "Price: %.2f rub.", items[i].price);
579         mvprintw(line++, 4, "Amount: %u", items[i].num);
580         line++;
581     }
582
583     mvprintw(rows - 2, 2, "Total: %u products. Press any key...", item_count)
584     ;
585     refresh();
586     getch();
587     endwin();
588 }
589
590 void show_database_info()
591 {
592     initscr();
593     clear();
594     refresh();
595
596     int rows, cols;
597     getmaxyx(stdscr, rows, cols);
598
599     box(stdscr, 0, 0);
600     mvprintw(0, 2, " DATABASE INFO ");
601
602     mvprintw(2, 4, "Signature: %.4s", header.signature);
603     mvprintw(3, 4, "Transaction: %u", header.tr_num);
604     mvprintw(4, 4, "Records in file: %u", header.st_count);
605     mvprintw(5, 4, "Records in memory: %u", item_count);
606     mvprintw(6, 4, "CRC-32: 0x%08X", header.crc32);

```

```

606     mvprintw(7, 4, "File: %s", filename);
607
608     mvprintw(rows - 2, 4, "Press any key to continue...");
609     refresh();
610     getch();
611     endwin();
612 }
613
614 void save_with_message(char *signature)
615 {
616     if (item_count == 0)
617     {
618         initscr();
619         clear();
620         refresh();
621         box(stdscr, 0, 0);
622         mvprintw(5, 10, "No data to save!");
623         mvprintw(7, 10, "Press any key...");
624         refresh();
625         getch();
626         endwin();
627         return;
628     }
629
630     header.st_count = item_count;
631     header = create_header(signature, header.tr_num, item_count, items);
632     save_database(filename, &header, items, item_count);
633
634     initscr();
635     clear();
636     refresh();
637     box(stdscr, 0, 0);
638     mvprintw(3, 10, "Database saved successfully!");
639     mvprintw(4, 10, "Records: %u", item_count);
640     mvprintw(5, 10, "CRC-32: 0x%08X", header.crc32);
641     mvprintw(7, 10, "Press any key...");
642     refresh();
643     getch();

```

```

644     endwin();
645 }
646
647 int main()
648 {
649     char *signature = "TASH";
650     int choice;
651     char status_msg[256] = "";
652     WINDOW *main_win;
653     int rows, cols;
654
655     // Загрузка базы данных
656     PRICE *loaded_data = NULL;
657     header = load_database(filename, &loaded_data);
658
659     if (loaded_data != NULL && header.st_count > 0)
660     {
661         items = loaded_data;
662         item_count = header.st_count;
663         snprintf(status_msg, sizeof(status_msg), "Loaded %u records.",
item_count);
664     }
665     else
666     {
667         memset(&header, 0, sizeof(DB_HEADER));
668         strncpy(header.signature, signature, 4);
669         header.tr_num = 0;
670         header.st_count = 0;
671         header.crc32 = 0;
672         snprintf(status_msg, sizeof(status_msg), "No database found. Creating
new.");
673     }
674
675     while (1)
676     {
677         initscr();
678         cbreak();
679         noecho();

```

```

680 keypad(stdscr, TRUE);
681 curs_set(0);
682 clear();
683 refresh();
684
685 getmaxyx(stdscr, rows, cols);
686
687 main_win = newwin(17, 55, (rows - 17) / 2, (cols - 55) / 2);
688 draw_main_menu(main_win, status_msg);
689
690 curs_set(1);
691 choice = get_menu_choice(main_win);
692
693 delwin(main_win);
694 endwin();
695
696 switch (choice)
697 {
698 case 1:
699 {
700     int added = input_data_ncurses();
701     if (added)
702     {
703         header.tr_num++;
704         snprintf(status_msg, sizeof(status_msg),
705                 "Product added. Transaction #%u", header.tr_num);
706         printf("Product added. Transaction #%u, CRC-32: 0x%08X",
header.tr_num, header.crc32);
707     }
708     else
709     {
710         snprintf(status_msg, sizeof(status_msg),
711                 "Product addition cancelled.");
712     }
713     break;
714 }
715
716 case 2:

```

```

717         if (item_count == 0)
718         {
719             snprintf(status_msg, sizeof(status_msg),
720                     "Database is empty.");
721             break;
722         }
723
724         int search_result = search_category_ncurses();
725         if (search_result == 1)
726         {
727             header.tr_num++;
728             snprintf(status_msg, sizeof(status_msg),
729                     "Search completed. Transaction #%u", header.tr_num);
730             printf("Search completed. Transaction #%u, CRC-32: 0x%08X",
header.tr_num, header.crc32);
731         }
732         else
733         {
734             snprintf(status_msg, sizeof(status_msg),
735                     "Search cancelled.");
736         }
737         break;
738
739     case 3:
740         show_all_products();
741         snprintf(status_msg, sizeof(status_msg),
742                 "Displayed all %u products.", item_count);
743         break;
744         printf("Displayed all %u products, CRC-32: 0x%08X", item_count,
header.crc32);
745
746     case 4:
747         save_with_message(signature);
748         snprintf(status_msg, sizeof(status_msg),
749                 "Database saved successfully.");
750         printf("Database saved successfully, CRC-32: 0x%08X", header.
crc32);
751         break;

```

```

752
753     case 5:
754         show_database_info();
755         snprintf(status_msg, sizeof(status_msg),
756                 "Database info displayed.");
757         printf("Database info displayed, CRC-32: 0x%08X", header.crc32);
758
759         break;
760
761     case 6:
762         if (item_count > 0)
763         {
764             header.st_count = item_count;
765             header = create_header(signature, header.tr_num, item_count,
766 items);
767             save_database(filename, &header, items, item_count);
768         }
769         for (unsigned int i = 0; i < item_count; i++)
770         {
771             free_price(&items[i]);
772         }
773         if (items)
774             free(items);
775
776         return 0;
777
778     default:
779         snprintf(status_msg, sizeof(status_msg),
780                 "Invalid choice. Please select 1-6.");
781         break;
782     }
783
784     return 0;
785 }

```

Листинг 13: Файл CmakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(PriceDB)
3
4 set(CMAKE_C_STANDARD 11)
5
6 set(SOURCES
7     src/main.c
8     src/price.c
9 )
10
11 add_executable(price_db ${SOURCES})
12
13 target_include_directories(price_db PRIVATE src)
14
15 find_package(ZLIB REQUIRED)
16
17 target_link_libraries(price_db
18     ncurses
19     form
20     panel
21     ${ZLIB_LIBRARIES}
22     m
23 )
```